SENAC · ADS CARUARU · 2026.2

CODING · LINGUAGENS E TÉCNICAS

Documentação do Projeto Final

O que cada equipe entrega além do código: três diagramas e um relatório técnico. Roteiro de cada artefato, exemplos prontos, erros que custam ponto e a rubrica usada na correção.

Domínio | Modelo ER | Arquitetura | Relatório

Entrega em 17 de novembro de 2026 · Prof. Sebastião Rogers

SUMÁRIO

O que tem neste documento

1 Como usar este documento

2 Diagrama de domínio

3 Modelo ER do banco

4 Diagrama de arquitetura em camadas

5 O relatório técnico

6 Formato de entrega

7 Erros que custam ponto

8 Avaliação

9 Prazo e checklist final

COMO LER OS BLOCOS

- ☐ Item de checklist: marque quando terminar.
- Código ou estrutura para copiar e adaptar.
- Exemplo resolvido: compare com o que você escreveu.
- Dica ou cuidado importante.

SEÇÃO 1

Como usar este documento

Segue o relatório com toda a documentação necessária para a entrega final do projeto da disciplina de Coding. O repositório mostra **o que** o sistema faz; esta documentação mostra **por que** ele é assim.

Coding é uma disciplina fullstack: requisitos, domínio, banco de dados, API, front-end e testes vivem no mesmo repositório. A documentação pedida aqui não é um documento de requisitos nem um projeto de banco de dados — é a visão geral do sistema inteiro, costurada de ponta a ponta.

Um critério governa tudo o que vem adiante: **o diagrama corresponde ao código que roda**. Não se exige notação UML impecável nem normalização justificada; exige-se que seja possível ler o diagrama, abrir o repositório e reconhecer o mesmo sistema. Por isso a documentação se escreve no fim, com o código aberto do lado, e não no começo, de memória.

Os quatro artefatos

Artefato	Responde à pergunta	Formato
Diagrama de domínio	Quais são as coisas que o sistema manipula e como se relacionam?	PNG + arquivo fonte
Modelo ER	Como essas coisas viraram tabelas no banco?	PNG + arquivo fonte
Diagrama de arquitetura	Por onde passa uma requisição, da tela ao banco e de volta?	PNG + arquivo fonte
Relatório técnico	Por que o projeto ficou desse jeito?	PDF, 8 a 12 páginas

Os três diagramas são a mesma história em três alturas: as ideias do negócio, o jeito como foram guardadas e o caminho que o dado percorre no código. O relatório justifica cada passagem de uma altura para a outra.

Fora de escopo: documento de visão, casos de uso formais, histórias de usuário com critério de aceite, dicionário de dados completo e diagrama de sequência.

DICA

Leia esta seção agora e volte às seções 2 a 5 na hora de montar cada artefato, uma por vez. A seção 7 reúne os erros que mais custam ponto e a seção 9 é a conferência da véspera. Todos os exemplos usam o mesmo sistema fictício: uma **biblioteca comunitária**, com Livro, Exemplar, Leitor e Empréstimo.

SEÇÃO 2

Diagrama de domínio

O domínio é o conjunto de coisas do mundo real que o sistema manipula, antes de existir banco, rota ou tela. Numa biblioteca: Livro, Exemplar, Leitor, Empréstimo. Numa oficina: Cliente, Veículo, Ordem de Serviço, Peça.

O que cada classe carrega

- o nome, no singular (`Emprestimo` , não `Emprestimos`);
- os atributos que importam, com o tipo (`data_devolucao: date`);
- os relacionamentos com multiplicidade marcada: `1` , `0..1` , `1..*` , `0..*` .

O que fica de fora

- `get` e `set` de cada atributo;
- classes de infraestrutura: `Base` , `Session` , `engine` , as rotas, os esquemas de validação;
- qualquer coisa que exista por causa do framework e não por causa do negócio.

Exemplo

Escrito em Mermaid. Cole em mermaid.live para ver o desenho — e o arquivo `.mmd` já serve como fonte editável.

```
classDiagram
    class Livro {
        isbn: str
        titulo: str
        autor: str
    }
    class Exemplar {
        codigo: str
        estado: str
    }
    class Leitor {
        matricula: str
        nome: str
    }
    class Emprestimo {
        data_retirada: date
        data_devolucao: date
    }
    Livro "1" --> "1..*" Exemplar
    Leitor "1" --> "0..*" Emprestimo
    Exemplar "1" --> "0..*" Emprestimo
```

Como se lê o exemplo

Um livro tem um ou mais exemplares físicos (`1` para `1..*`); um leitor pode ter nenhum ou vários empréstimos (`1` para `0..*`); cada empréstimo é de exatamente um exemplar.

EXEMPLO DE DECISÃO

A separação entre **Livro** e **Exemplar** não é detalhe de modelagem: a biblioteca empresta o objeto físico, não o título. Duas cópias do mesmo ISBN circulam de forma independente — uma pode estar emprestada enquanto a outra está na estante. Decisões desse tipo são exatamente o que o relatório técnico precisa justificar.

Atalho para começar

Pegue o enunciado do seu projeto e sublinhe os substantivos: a maioria das classes está ali. Depois, para cada par de classes, pergunte "uma precisa da outra para existir?". As respostas são os seus relacionamentos.

Antes de entregar

- [ ] Toda classe do diagrama existe no código, com o mesmo nome.
- [ ] Toda classe de negócio do código aparece no diagrama.
- [ ] Todo relacionamento tem multiplicidade marcada nas duas pontas.
- [ ] Nenhuma classe de infraestrutura entrou por engano.

CUIDADO

Diagrama de domínio não é diagrama de classes do código. Se aparecerem `LivroSchema` , `LivroCreate` e `LivroResponse` como três classes diferentes, o que está desenhado é o framework, não o negócio.

SEÇÃO 3

Modelo ER do banco

A fonte da verdade aqui são os seus modelos SQLAlchemy. O modelo ER é a tradução em desenho do que está declarado no código: se a coluna não existe na classe, não existe no diagrama.

O que cada tabela carrega

- o nome exato do `__tablename__`;
- as colunas com o tipo do banco: `INTEGER`, `VARCHAR(120)`, `DATE`, `BOOLEAN`;
- a chave primária marcada (PK, de *primary key*);
- as chaves estrangeiras marcadas (FK, de *foreign key*), apontando para a tabela de destino;
- a cardinalidade de cada relacionamento.

Exemplo: do modelo para a tabela

```python
class Emprestimo(Base):
    __tablename__ = "emprestimo"
    id             = Column(Integer, primary_key=True)
    exemplar_id    = Column(Integer, ForeignKey("exemplar.id"), nullable=False)
    leitor_id      = Column(Integer, ForeignKey("leitor.id"), nullable=False)
    data_retirada  = Column(Date, nullable=False)
    data_devolucao = Column(Date, nullable=True)
```

No diagrama, essa classe vira exatamente isto:

```
emprestimo
  id               INTEGER   PK
  exemplar_id      INTEGER   FK → exemplar.id    NOT NULL
  leitor_id        INTEGER   FK → leitor.id      NOT NULL
  data_retirada    DATE      NOT NULL
  data_devolucao   DATE      NULL
```

Cada linha do código tem uma linha correspondente no desenho. É essa correspondência, coluna por coluna, que será conferida na correção.

Exemplo: muitos para muitos

Um livro pode ter várias categorias e uma categoria, vários livros. No domínio isso é uma linha entre duas classes. No banco, são **três** tabelas:

```
livro                          categoria
  id       INTEGER  PK           id      INTEGER  PK
  titulo   VARCHAR(200)         nome    VARCHAR(60)
```

E, entre as duas, a tabela de associação — a que o código cria e quase todo mundo esquece de desenhar:

```
livro_categoria
  livro_id       INTEGER  PK, FK → livro.id
  categoria_id   INTEGER  PK, FK → categoria.id
```

A chave primária dessa tabela é o par das duas colunas: é isso que impede o mesmo livro de receber a mesma categoria duas vezes.

Domínio e ER não são o mesmo desenho

No domínio existem herança, composição e objetos dentro de objetos. No banco relacional existem tabela, coluna e chave estrangeira — nada mais. Toda ideia do domínio que não cabe nesses três elementos precisa ser convertida, e a conversão é explicada no relatório técnico.

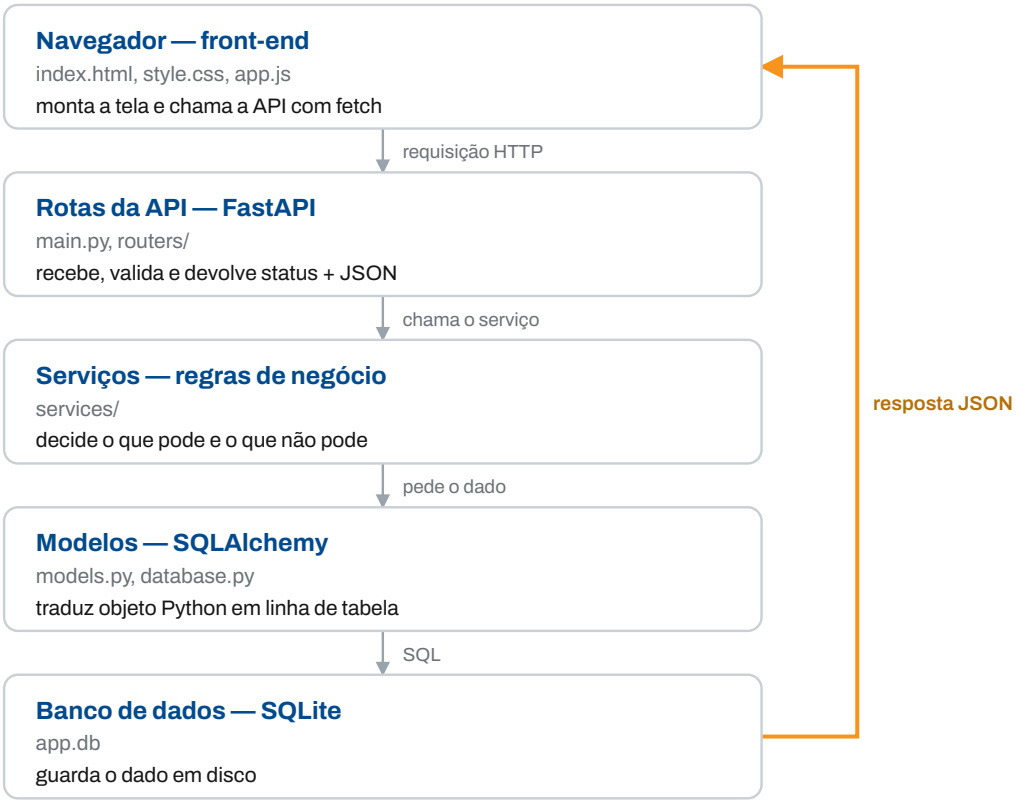
Erros frequentes

Erro	O que fazer
Muitos para muitos como linha direta	desenhar a tabela de associação, que existe no banco
Herança desenhada com seta de herança	o banco não tem isso: diga se virou uma tabela com coluna de tipo, duas tabelas ou três — e por quê
Coluna FK sem destino	marcar `FK → tabela.coluna` em cada uma
Tabela que o `create_all` nunca criou	remover do diagrama ou criar no código

SEÇÃO 4

Diagrama de arquitetura em camadas

Este diagrama mostra o caminho de uma requisição: do clique na tela até a linha gravada no banco, e a volta. Cada caixa leva o nome real do arquivo ou da pasta do **seu** projeto.



Exemplo do nível de detalhe esperado. As setas descendo são o pedido; a seta laranja é a resposta, que sobe pelas mesmas camadas na ordem inversa.

Exemplo de leitura: um POST /emprestimos

1. `app.js` monta o corpo e chama `fetch("/emprestimos", { method: "POST" })`.
2. `routers/emprestimos.py` recebe, valida o corpo e chama o serviço.
3. `services/emprestimo.py` verifica se o exemplar está disponível e se o leitor não passou do limite.
4. `models.py` grava a linha na tabela `emprestimo`.
5. A resposta sobe: `201 Created` com o JSON do empréstimo criado.

A regra que o desenho precisa deixar visível

Cada camada só conversa com a vizinha. A rota não escreve SQL. O modelo não devolve JSON. O navegador não sabe que existe SQLite do outro lado.

> **HONESTIDADE VALE PONTO**
>
> Se no seu código a rota fala direto com o banco, desenhe assim. Vale mais um diagrama honesto com quatro camadas do que um diagrama bonito que mente sobre cinco — e a seção "o que ficou de fora" do relatório é o lugar de explicar por que ficou assim.

Antes de entregar

- [ ] Cada caixa tem o nome real de um arquivo ou pasta do projeto.
- [ ] O caminho de ida e o de volta estão desenhados.
- [ ] Nenhuma seta pula uma camada sem que isso seja verdade no código.

Se quiser desenhar em Mermaid

Ponto de partida para adaptar com os nomes do seu projeto. O arquivo `.mmd` já é a fonte editável, e o GitHub renderiza o desenho direto no README.

```
flowchart TD
  A["Navegador — index.html, app.js"] --> B["Rotas — main.py, routers/"]
  B --> C["Serviços — services/"]
  C --> D["Modelos — models.py"]
  D --> E[("SQLite — app.db")]
  E -.resposta JSON.-> A
```

SEÇÃO 5

O relatório técnico

De 8 a 12 páginas, em PDF. Onze seções curtas — alguns parágrafos cada, nenhuma passa de uma página.

#	Seção	O que entra
1	**Identificação**	nome do projeto, integrantes, link do repositório
2	**O problema e o cliente**	que dor o sistema resolve e para quem, em um parágrafo
3	**O que o sistema faz hoje**	o que está funcionando no repositório, não o que foi planejado em setembro
4	**Decisões de modelagem**	por que essa entidade existe, por que herança aqui e composição ali
5	**Do domínio ao banco**	o que mudou na tradução: a herança e o muitos para muitos entram aqui
6	**A API**	uma tabela de endpoints, sem prosa descrevendo rota por rota
7	**As camadas**	quem chama quem e o que cada camada **não** faz
8	**O front-end**	quais telas existem e como falam com a API
9	**Testes**	o que está coberto e o que não está
10	**O que ficou de fora**	limites reconhecidos valem ponto; limites escondidos custam
11	**Como rodar**	os comandos na ordem, do `git clone` ao servidor de pé

Sobre a escrita: frase curta, voz ativa, sem "o presente trabalho tem por objetivo". É um documento técnico escrito para outro programador que vai pegar o projeto e precisa se virar sozinho.

Exemplo: uma decisão de modelagem

FRACO

"Criamos a classe Exemplar para representar os exemplares dos livros."

BOM

"Separamos **Livro** de **Exemplar** porque a biblioteca empresta o objeto físico, não o título: duas cópias do mesmo ISBN circulam de forma independente. Sem a separação, não haveria como registrar que a cópia 2 está emprestada enquanto a 1 está na estante."

A diferença entre as duas versões: a segunda diz o que quebraria se a decisão fosse outra. É isso que conta como justificativa.

Exemplo: a tabela de endpoints

Método	Rota	O que faz	Sucesso
GET	`/livros`	lista os livros, com filtro opcional por título	200
POST	`/emprestimos`	registra o empréstimo de um exemplar	201
PATCH	`/emprestimos/{id}/devolucao`	registra a devolução	200
DELETE	`/livros/{id}`	remove um livro sem exemplares emprestados	204

CONFIRA NA PRÓPRIA APLICAÇÃO

Suba o projeto e abra `/docs` : o FastAPI lista as rotas que realmente existem, com método e código de status. A tabela do relatório tem que bater com aquela página, não com o plano de setembro.

SEÇÃO 6

Formato de entrega

Tudo vai dentro do próprio repositório, numa pasta `docs/`. Entrega por e-mail, WhatsApp ou link de Drive não é considerada: o que não está no repositório não foi entregue.

```
seu-projeto/
├─ docs/
│  ├─ dominio.png
│  ├─ dominio.drawio
│  ├─ er.png
│  ├─ er.drawio
│  ├─ arquitetura.png
│  ├─ arquitetura.drawio
│  └─ relatorio.pdf
└─ README.md
```

O `README.md` da raiz precisa linkar os quatro arquivos — a correção começa por ele.

Por que a fonte editável junto com a imagem

PNG apodrece em silêncio. Daqui a um mês vocês mudam uma entidade, abrem a imagem para corrigir e não conseguem; aí o diagrama fica desatualizado para sempre. Com o arquivo fonte, corrigir leva dois minutos.

Ferramentas sugeridas

Ferramenta	Boa para	Observação
draw.io	os três diagramas	salva `.drawio` e exporta PNG; roda no navegador
dbdiagram.io	modelo ER	você escreve as tabelas em texto e ele desenha
Mermaid	domínio e arquitetura	o diagrama é código: versiona no Git junto com o projeto e o GitHub renderiza no README
Excalidraw	arquitetura	traço de rascunho, bom quando o desenho é para explicar

SEÇÃO 7

Erros que custam ponto

Os problemas mais frequentes nesta entrega, o que cada um revela para quem corrige e como resolver em poucos minutos.

Erro	O que revela	Correção
Muitos para muitos sem a tabela de associação	o diagrama foi desenhado de memória, não a partir do banco	abrir `models.py`, localizar a tabela de associação e desenhar as três
Classe no diagrama que não existe no código	o diagrama parou na versão de setembro	refazer a partir do código de hoje
Chave estrangeira sem destino	a FK foi copiada do domínio sem olhar a tabela	marcar `FK → tabela.coluna` em cada uma
Arquitetura com caixas genéricas: Front, Back, Banco	ninguém leu o próprio projeto	trocar pelos nomes reais de arquivo e pasta
Relatório que descreve o código em vez de justificar	confunde documentação com comentário de código	em cada decisão, escrever o que quebraria se fosse diferente
Tabela de endpoints com rotas que não existem	copiada do plano, não da aplicação	subir o projeto e conferir em `/docs`
"Como rodar" que não roda	nunca foi testado numa pasta limpa	clonar o próprio repositório em outra pasta e seguir o passo a passo
Só o PNG, sem a fonte editável	o diagrama não poderá mais ser corrigido	exportar também o `.drawio`, ou guardar o `.mmd`
PDF que é um `.docx` renomeado	pressa na última hora	exportar de verdade para PDF e abrir o arquivo antes de subir

Nenhum desses erros exige refazer o projeto: todos são de conferência. É por isso que a seção 9 existe.

SEÇÃO 8

Avaliação

A documentação vale 10 pontos e compõe a nota do Projeto Final, junto com o código e a apresentação.

Critério	Pontos	O que separa a nota cheia da metade
Diagrama de domínio	**2,0**	classes, atributos e multiplicidade completos · contra caixas soltas sem relacionamento
Modelo ER	**2,0**	PK, FK, tipos e tabela de associação no lugar · contra tabelas sem chave e muitos para muitos sem o meio
Diagrama de arquitetura	**1,5**	nomes reais de arquivo e o caminho de ida e volta · contra três caixas genéricas empilhadas
Relatório: decisões	**2,5**	cada escolha explicada com o porquê · contra descrição do que o código faz
Relatório: API, testes e como rodar	**1,0**	tabela de endpoints correta e passo a passo que funciona · contra comandos que não sobem o projeto
Correspondência com o código	**1,0**	diagrama e repositório contam a mesma história · contra entidade que só existe no desenho

A última linha vale 1 ponto direto, mas o efeito indireto é maior: se o diagrama não bate com o código, os critérios de cima caem junto. Não adianta caprichar no desenho de um sistema que não foi escrito.

E o contrário também vale: um projeto pequeno, bem documentado e honesto sobre os próprios limites tira mais nota aqui do que um projeto ambicioso com documentação de fachada.

SEÇÃO 9

Prazo e checklist final

O último `push` precisa estar no repositório até as **18h de 17 de novembro de 2026**, dia da apresentação do Projeto Final. A leitura da documentação acontece durante as apresentações, então commits posteriores ao horário não entram na correção.

Passem por esta lista na véspera. São os erros que mais custam ponto e todos levam menos de dez minutos para corrigir.

- [ ] Abri o repositório numa aba anônima e os quatro arquivos aparecem.
- [ ] Os links do README funcionam, todos.
- [ ] Cada diagrama tem o PNG **e** o arquivo fonte.
- [ ] Toda classe do diagrama de domínio existe no código, com o mesmo nome.
- [ ] Toda tabela do ER foi realmente criada pelo código.
- [ ] Os muitos para muitos têm a tabela de associação desenhada.
- [ ] O diagrama de arquitetura usa os nomes de arquivo do nosso projeto.
- [ ] A tabela de endpoints bate com as rotas que existem em `/docs`.
- [ ] Segui o meu próprio "como rodar" numa pasta limpa e o projeto subiu.
- [ ] O relatório diz o que ficou de fora.
- [ ] O PDF abre e não é um `.docx` renomeado.
- [ ] O nome de todo mundo da equipe está na primeira página.

Para ir além

- **Documentação do draw.io** — como exportar PNG mantendo o arquivo fonte.
- **Sintaxe de classes no Mermaid** — para o diagrama de domínio versionado junto com o código.
- **Sintaxe ER no Mermaid** — a alternativa ao dbdiagram.io.
- **FastAPI: documentação automática em /docs** — a página que confere a sua tabela de endpoints.
- **SQLAlchemy: relacionamentos** — incluindo o muitos para muitos com tabela de associação.

ÚLTIMA PALAVRA

Documentação não é enfeite de entrega. É o que permite que outra pessoa — ou vocês mesmos, daqui a seis meses — abram este projeto e entendam as decisões sem ter que ler linha por linha. Escrevam para essa pessoa.

Documentação do Projeto Final · Coding: Linguagens e Técnicas · CST em Análise e Desenvolvimento de Sistemas · Faculdade Senac Pernambuco, unidade Caruaru · semestre 2026.2 · Prof. Sebastião Rogers.